

Krefasy Backoffice — Guia Técnico para Desenvolvedores

Versão: 1.0 | Data: Julho 2026 | Público: Novos desenvolvedores

1. Visão geral

O **Krefasy Backoffice** é uma aplicação web administrativa para gestão de créditos e empréstimos. Permite que administradores gerem clientes, empréstimos, parcelas, configurações de produtos financeiros e atendimento por chat.

Item	Detalhe
Nome do projeto	backoffice-krefasy--app
Template base	Visto Vue (customizado para Krefasy)
API backend	https://credit-api.krefasy.com/api/v1
Acesso	Apenas utilizadores com role Admin

2. Stack tecnológica

Tecnologia	Uso
Vue 3 + TypeScript	Framework UI (Composition API, <script setup>)
Vite 5	Bundler e dev server
Vue Router 4	Routing com lazy loading
Pinia	Gestão de estado
Tailwind CSS 3	Estilização utilitária
Axios	Cliente HTTP
ApexCharts	Gráficos no dashboard
SweetAlert2	Diálogos de confirmação/erro
vue-i18n	Internacionalização

3. Estrutura de pastas

```
backoffice-krefasy--app/
├─ public/                # Assets estáticos
├─ src/
│   ├─ assets/css/        # Estilos globais + krefasy-theme.css
│   ├─ components/layout/  # Header, Sidebar, Footer, PageHeader
│   ├─ composables/        # Lógica reutilizável
│   ├─ layouts/            # app-layout, auth-layout
│   ├─ router/index.ts     # Rotas
│   ├─ services/           # Camada API (um ficheiro por domínio)
│   ├─ stores/             # Pinia (app + krefasy)
│   ├─ utils/
│   └─ views/              # Páginas
├─ docs/                  # Documentação
├─ env.example
├─ Dockerfile
└─ package.json
```

Nota: O projeto herda ~70 rotas de demonstração do template Vristo (`/apps/*` , `/elements/*` , etc.) que **não fazem parte do produto Krefasy**. O menu ativo está em `Sidebar.vue` .

4. Arquitetura em camadas

Views (pages)	→ <code>krefasy-chat.vue</code> , <code>customers/index.vue</code>
Components	→ <code>PageHeader</code> , <code>Modals</code>
Stores (Pinia)	→ <code>krefasy.store.ts</code>
Services	→ <code>messages.service.ts</code> , <code>loans.service.ts</code>
API base (Axios)	→ <code>api.ts</code> (JWT automático)
Backend .NET	→ <code>credit-api.krefasy.com</code>

Fluxo: View → Store/Service → `api.ts` → Backend → resposta `{ succeeded, data, message, errors }` → Store atualiza refs → View re-renderiza.

5. Services (camada API)

Ficheiro	Domínio
<code>api.ts</code>	Cliente HTTP base, interceptors JWT
<code>auth.service.ts</code>	Login, logout, sessão
<code>customers.service.ts</code>	Clientes

<code>loans.service.ts</code>	Empréstimos
<code>parcels.service.ts</code>	Parcelas
<code>messages.service.ts</code>	Chat / conversas
<code>users.service.ts</code>	Utilizadores admin
<code>currencies.service.ts</code>	Moedas
<code>countries.service.ts</code>	Países
<code>loan-products.service.ts</code>	Produtos de empréstimo
<code>loan-status.service.ts</code>	Status de empréstimo
<code>interest-periods.service.ts</code>	Períodos de juros
<code>payment-method-types.service.ts</code>	Métodos de pagamento

6. Autenticação

- **Login:** `/auth/login` → `POST /auth/login`
- **Sessão:** `localStorage` chave `USER_LOGIN` (token + user)
- **Rotas protegidas:** `meta.requiresAuth: true` no router
- **Interceptor:** adiciona `Authorization: Bearer` ; 401 → redirect login
- **Logout:** `krefasyStore.logout()`

7. Integração API (.NET)

O backend usa **PascalCase** em query params e multipart. O frontend usa camelCase; os services fazem o mapeamento.

Frontend	API (.NET)
<code>page</code>	<code>Page</code>
<code>limit</code>	<code>Limit</code>
<code>unreadOnly</code>	<code>UnreadOnly</code>
<code>conversationId</code> (FormData)	<code>ConversationId</code>

Envelope de resposta: `{ "succeeded": true, "data": {...}, "message": "...", "errors": null }`

8. Páginas Krefasy (produto ativo)

Menu principal

Rota	Página	Descrição
<code>/dashboard</code>	<code>krefasy-dashboard.vue</code>	KPIs e gráficos
<code>/customers</code>	<code>customers/index.vue</code>	CRUD clientes
<code>/customers/:id</code>	<code>customers/detail.vue</code>	Detalhe cliente
<code>/loans</code>	<code>krefasy-loans.vue</code>	Listagem empréstimos
<code>/loans/view/:id</code>	<code>krefasy-loan-detail.vue</code>	Detalhe empréstimo
<code>/loans/pending</code>	<code>krefasy-loans-pending.vue</code>	Pendentes aprovação

/chat	krefasy-chat.vue	Chat com clientes (API real)
/users	users/index.vue	Utilizadores admin
/reports	krefasy-reports.vue	Relatórios

Configurações (submenu)

Rota	Descrição
/loan-products	Produtos de empréstimo
/currencies	Moedas
/countries	Países
/interest-periods	Períodos de juros
/loan-status	Status de empréstimos
/payment-method-types	Métodos de pagamento
/loan-interest-rates	Taxas de juros

9. Módulo Chat

Ficheiro	Função
krefasy-chat.vue	UI principal do chat
ConversationModal.vue	Criar conversa (admin)
messages.service.ts	API client com mapeamento PascalCase
use-chat-notifications.ts	Polling de não lidas (30s)

Notificações: badge na Sidebar + sino no Header com dropdown. Ao clicar, abre /chat na conversa certa.

10. Gestão de estado (Pinia)

Store	Responsabilidade
useAppStore	Tema, sidebar, layout, i18n

useKrefasyStore

Dados de negócio: clientes, empréstimos, chat, dashboard

11. Design system Krefasy

Definido em `src/assets/css/krefasy-theme.css` :

Token	Valor	Uso
<code>--krefasy-navy</code>	<code>#0e1133</code>	Header, fundos escuros
<code>--krefasy-purple</code>	<code>#801f82</code>	Cor primária
<code>--krefasy-surface-muted</code>	—	Fundo das páginas

Componentes: `PageHeader` (título + actions), `Sidebar` , `Header` .

12. Variáveis de ambiente

Variável	Descrição
<code>VITE_API_BASE_URL</code>	URL base da API
<code>VITE_MAX_FILE_SIZE</code>	Limite upload (10MB default)
<code>VITE_WEBSOCKET_URL</code>	WebSocket (fase 2)

Nunca colocar secrets em variáveis `VITE_*` — são expostas no bundle.

13. Setup local

```
yarn install
cp env.example .env
yarn dev      # http://localhost:5173
yarn build    # gera dist/
yarn preview  # preview do build
```

14. Deploy (Docker)

```
yarn docker:build
yarn docker:push:hub
```

O `Dockerfile` serve ficheiros estáticos de `dist/` via nginx.

15. Convenções

Tipo	Convenção	Exemplo
Views Krefasy	krefasy-*.vue ou index.vue	krefasy-loans.vue
Services	*.service.ts	loans.service.ts
Modals	*Modal.vue	CustomerModal.vue
Composables	use-*.ts	use-chat-notifications.ts

- Toda chamada HTTP passa por `src/services/`
- Erros nas views com SweetAlert2
- Novas páginas usam `PageHeader`
- Não commitar `.env`

16. Recursos

Recurso	Localização
API Swagger	credit-api.krefasy.com
README	README-KREFASY.md
Env exemplo	env.example
Este guia (MD)	docs/ONBOARDING-DESENVOLVEDORES.md